# Automating the Management of Software Systems

DRAFT [in progress]

## Background

Modern software systems are vast, often spanning thousands of interconnected services and components distributed across data centers and geographic regions. These systems process enormous volumes of requests and evolve continuously through frequent deployments. In such environments, failures or disruptions are costly and degrade the user experience, necessitating the development of intelligent tools that can automatically manage software systems to ensure their reliability, performance and resilience.

Traditionally, to understand the execution of a software system, the engineer relied on log files to provide clues about what the system was doing during execution. Typically, this was in the form of unstructured logs and it could include a stack trace provided by an exception. These clues would then be used to understand the behavior of the design that led to the unwanted state. In some cases, the bug is the result of an incorrect implementation of the design and in other cases, the design learns about constraints imposed by the environment that it was unaware of. While this is already a tedious process, the distributed nature of modern software systems adds to the challenge. There have been many interesting approaches to expedite the debugging and recovery process by understanding the behavior of the design from the logs but all these solutions inherently suffer from the same limitation, they are working with incomplete information and as a result, their conclusions are inherently probabilistic instead of verifiable facts.

To address these limitations, this document proposes an approach that grounds the understanding of execution in a faithful, semantic representation of the system's design. This representation enables execution to be interpreted deterministically, providing a foundation for automated analysis and management without reliance on probabilistic inference.

## Design Semantics and Behavioral Correctness

To understand the correctness of a software system, it is necessary to define how execution is understood through the behavior of its design. In this context, behavior is defined as the actions taken and choices made by the design in response to a given input.

The design unambiguously establishes the intentions of a world governed by semantic relationships and constraints, expressed as *semantic invariants* that define the conditions under

which the world is valid. These invariants constitute the authoritative definition of correctness for the design.

When the design's behavior preserves all semantic invariants, the system remains in a *semantically valid state* and functions as intended. When one or more invariants are violated, the system enters a *semantically invalid state*, regardless of whether execution continues, degrades, or terminates.

## Semantic Invariants: Prediction and Resolution of Invalid States

Semantic invariants establish the correctness of a design's behavior by defining the conditions under which its intentions can be realized. Since these invariants specify what must always hold true for the design's behavior to be correct, they enable the design itself to predict semantically invalid states. When an invariant is violated, the resolution occurs at the design level by refining the design's behavior to enforce the invariant, thereby restoring semantic validity.

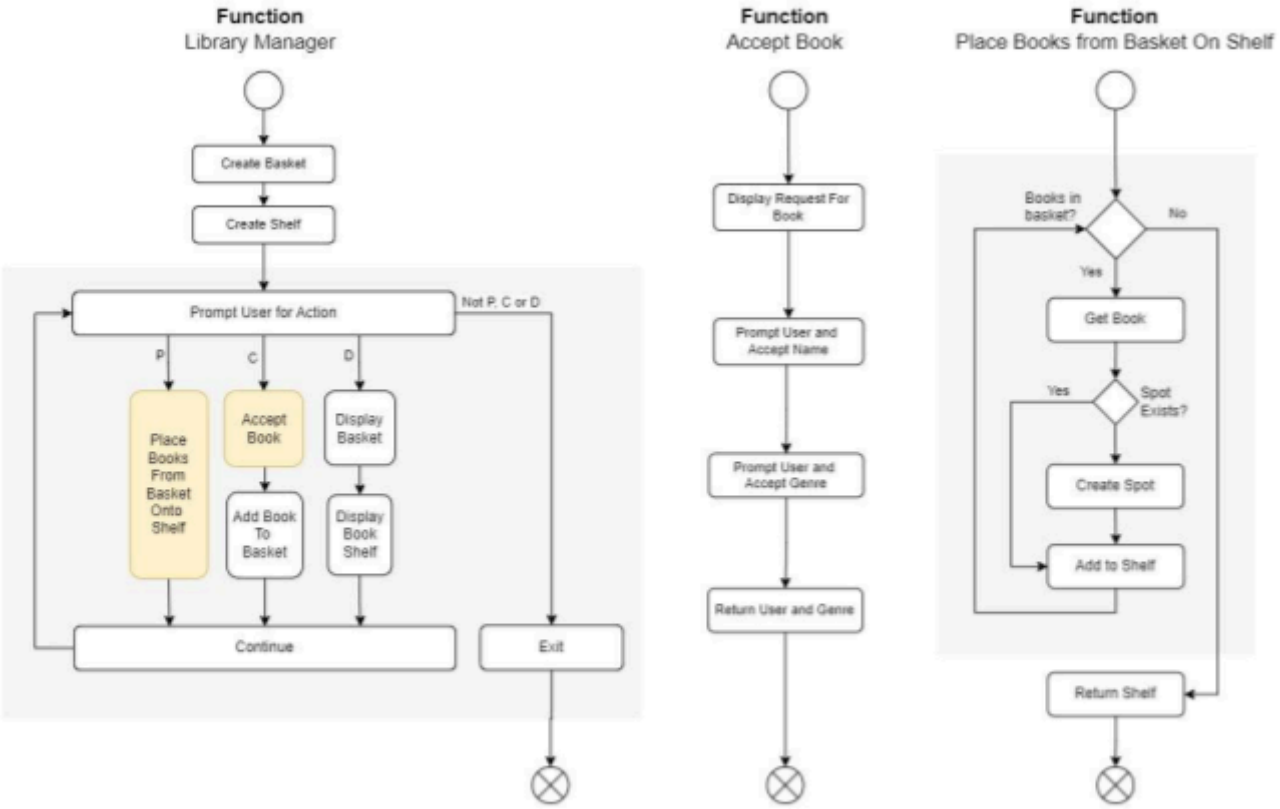



Figure 1: Behavioral semantics of the design of a library manager.

Semantic invariants are not arbitrary constraints imposed on a system after the fact; they emerge directly from the structure of the design and the semantics of its behavior. Figure 1 illustrates this using the library manager design, where invariants arise naturally from the design's control flow, data dependencies, and semantic assumptions.

For example, in the library manager shown in Figure 1, the design accepts books and places them on shelves using the first letter of each book's name as a sorting key. This behavior implicitly assumes that every accepted book has a name. Accepting a book without a name therefore violates a semantic invariant and allows the design to predict a semantically invalid state when the behavior attempts to read the first letter to check if the slot exists on the shelf. The resolution occurs at the design level through a behavioral refinement: rejecting books without a name at the point of acceptance. By enforcing this invariant, the design prevents the system from entering a semantically invalid state.

## Environmental Conditions and Learned Invariants

However, the assumptions made by a design can conflict with the reality of what is actually possible. A design encodes expectations about its operating environment, and those expectations may be refuted by observed behavior during execution. When this occurs, the environment imposes restrictions on how the design can realize its intentions, forcing the design to learn new behavior and introduce new constraints.

For example, in the library manager, the design assumes that it can take all the books in the basket to the shelf and place them on the shelf. However, for example, in practice, if the basket contains more than five books, it cannot be carried to the bookshelf because it becomes too heavy. The environment therefore introduces a previously unknown constraint. To continue realizing its intention of placing books on the shelf, the design must adapt its behavior by only carrying up to five books at a time.

Through this process, the design acquires new, evidence-backed constraints derived from execution. These constraints both guide future behavior and serve as provable conditions under which the design's intentions can be realized, transforming environmental restrictions into explicit knowledge embedded within the design itself.

# Design Driven Automation

In software systems, a design is realized through an implementation in a programming language, which uses its abstractions to enact the design's intentions while enforcing the semantic invariants that define correctness. When the implementation is executed and the system enters a semantically invalid state, debugging can be understood as interpreting the execution through the behavior of the design in order to restore alignment with its intentions. Because semantic invariants are unambiguous, the root cause of an invalid state can be identified by determining how the implementation violated one or more invariants. If no known invariant violation can be identified, the invalid state instead reveals a limitation imposed by the environment, enabling the design to learn and incorporate new invariants or behavioral refinements. This process eliminates probabilistic diagnosis, every semantically invalid state is either explained by an invariant violation or results in new design semantics.

To address the limitations of traditional techniques, this document presents an approach that automatically interprets the execution of a software system through a faithful representation of the design's behavior. Since execution is understood in terms of semantic invariants, this approach has several direct implications.

- First, it automates debugging by deterministically identifying which semantic invariants were violated when the system enters a semantically invalid state.
- Second, it automates testing, since an implementation that respects all semantic invariants is, by definition, capable of realizing the design's intentions.
- Third, it automates systems management by continuously ensuring that execution remains aligned with the design's intentions in its operational environment.

Practically, to achieve this automation, two abilities must be developed:

1. The design must be unambiguously instrumented and the implementation must be mapped onto the behavioral abstractions of the design. The execution must be transformed into the behavior through the mapping and the behavior must be validated through the semantic invariants of the design.
2. The behavior of the design and the environment that triggered the behavior must be losslessly preserved. The biggest hurdle is the sheer volume of the data and to address this, the unambiguous nature of the design can be exploited to apply domain specific compression to the data.

## Lossless Behavior and Domain-Specific Compression

The automation is only possible if the behavior of the design is retained losslessly. Any gaps in behavioral representation break the automatic nature of the process, as execution can no longer be understood purely through observation. Instead, missing behavior must be inferred, reintroducing probability and uncertainty into the analysis.

The same requirement applies to testing and learning. If the environments that motivated semantic invariants cannot be unambiguously identified from execution, then testing cannot be automated and constraints cannot be reliably validated. For this reason, it is critical that execution is preserved losslessly.

To make this feasible, the unambiguous nature of the design is leveraged to fully specify the structure of the execution data. Since the design defines exactly what behavior is possible, this structure can be exploited to apply domain-specific compression, significantly reducing the size of the resulting execution trace. Since program execution is highly repetitive and predictable, this compression can be further improved by leveraging recurring structural patterns in the data.

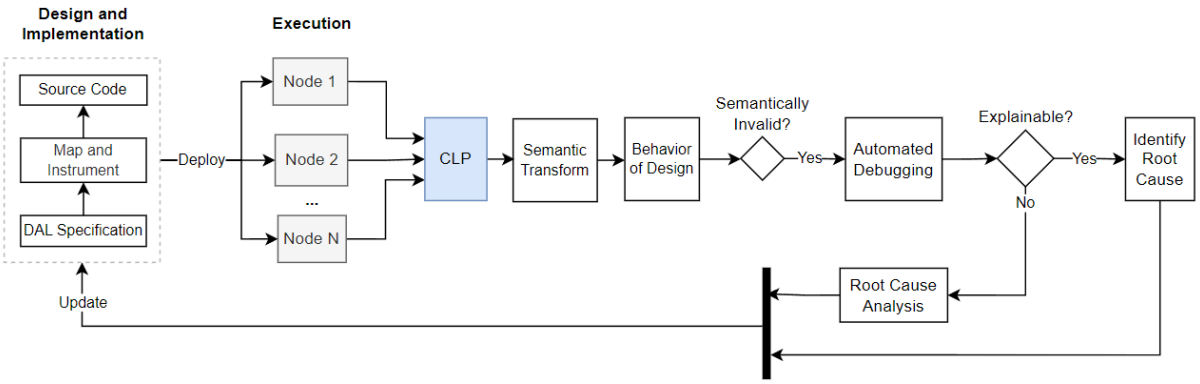
Through this process, the complete execution history and a lossless representation of design behavior can be preserved in a practical and scalable form. To manage these domain-compressed execution traces, an open-source tool named CLP can be used. CLP supports domain-specific compression of unambiguous dynamic traces and enables efficient search directly over compressed data, without requiring decompression. It has been proven at petabyte scale, making it a practical foundation for enabling automated debugging, testing, and systems management.

## Design Instrumentation and Semantic Transformation

To make this possible, the design is unambiguously instrumented into the implementation using a Design Abstraction Language (DAL), such that the abstractions of the design are mapped directly onto the implementation. As a result, the execution of the software system can be transformed into the behavior of the design, allowing execution to be automatically understood through the design’s semantics. This transformation is referred to as a Semantic Transform (ST).

For designs that involve concurrency, the Semantic Transform maps execution distributed across multiple threads, processes, or nodes into the behavior of a single, coherent design. This is achieved by propagating unique identifiers across threads, processes, and nodes, allowing the transform to maintain continuity of design behavior across distributed execution. By interpreting distributed execution through a single design, the transform collapses incidental complexity and enables deterministic understanding of distributed system behavior.

## The Design Feedback Loop




Through instrumentation, domain-specific compression, and semantic transformation, the entire process is encapsulated in the feedback loop illustrated above. The design serves as the highest authority over the software system, ensuring that execution is always understood with semantic clarity rather. When execution enters a semantically invalid state, the behavior of the design is examined directly to determine whether the cause can be explained by an existing invariant violation. If no such explanation exists, the process enables the design to unambiguously learn from its environment by incorporating new invariants or behavioral refinements.

Through this loop, both the evolution of the design and the realities that shaped it are preserved losslessly. The design does not merely guide implementation; it continuously audits execution, validates its own assumptions, and evolves in response to observed reality, without ever dissolving into ambiguity or probabilistic reasoning.

# The Design Learning Platform

Traditional observability platforms were built to compensate for incomplete understanding. They collect diagnostic data from logs to infer what a design might have been doing during execution. Since the design's intentions and semantics are not explicitly represented, these platforms rely on techniques like correlation, and probabilistic reasoning to reconstruct behavior after the fact. As a result, debugging, testing, and systems management remain reactive, manual, and fundamentally uncertain.

The approach presented in this document establishes a fundamentally different foundation. In this solution, the design behavior can be observed instead of inferred. Execution is interpreted through the design itself, allowing valid and invalid semantic states to be identified deterministically. This eliminates the need for speculative analysis and replaces it with validation against an authoritative semantic model.

Through lossless preservation of behavior and environment, the system retains not only what happened, but why it happened. Each semantically invalid state is either explained by an existing invariant violation or results in the introduction of new invariants or behavioral refinements. In this way, the design continuously learns from execution, incorporating validated knowledge about both its own assumptions and the constraints imposed by its operating environment.

Together, these capabilities form a **design learning platform**. In this platform, the design is a living semantic model that audits execution, validates its own correctness, and evolves in response to observed reality in an unambiguous way. Since learning is grounded in lossless, semantically meaningful execution data, this evolution is fully explainable, auditable, and reproducible.

This platform is also *fully optimized*. Since the design unambiguously defines what behavior is possible, all data collection, storage, and analysis are strictly bounded by semantic necessity. There is no waste in storage or computation as everything is driven by the unambiguous design and its intentionally designed world. Domain-specific compression exploits the repetitive and predictable structure of design, ensuring that execution history is preserved at minimal cost while remaining fully searchable and lossless. In addition, CLP enables the system to be searched without decompression, further optimizing the platform's efficiency.

In this model, debugging becomes the deterministic identification of violated invariants. Testing becomes the validation that execution preserves all known semantic invariants under defined environments. Systems management becomes the continuous enforcement of alignment between reality and design intent. By eliminating ambiguity at the source, the design learning platform removes the very inefficiencies that traditional observability and management tools were created to mitigate.